

LAPORAN UJI KEAMANAN SISTEM

Kelompok 03 – Topik A: Web Server dengan Docker

Pendahuluan

Dokumen ini merupakan laporan hasil pengujian keamanan terhadap sistem web server yang telah di-deploy menggunakan Docker pada VM Ubuntu [cite: 1]. Pengujian dilakukan untuk memverifikasi bahwa setiap lapisan keamanan yang diimplementasikan berfungsi dengan benar sesuai dengan rubrik penilaian mata kuliah Keamanan Sistem dan Jaringan (KSJ) [cite: 1].

Lingkungan Pengujian

Komponen	Detail
Sistem Operasi	Ubuntu 22.04 LTS (VM) [cite: 1]
IP Server	172.30.67.106 (Eduroam) / 100.110.137.96 (Tailscale) [cite: 1]
Docker Engine	Docker CE + Docker Compose Plugin [cite: 1]
Web Server	Nginx (Alpine) sebagai Reverse Proxy [cite: 1]
App Server	PHP-FPM (Laravel) [cite: 1]
Database	MySQL 8.0 [cite: 1]

Catatan: Akses pengujian dilakukan melalui Tailscale (IP 100.110.137.96) karena server berada di jaringan kampus yang memerlukan koneksi Eduroam [cite: 1]. Tailscale membuat tunnel VPN langsung ke VM sehingga pengujian dapat dilakukan dari jaringan mana pun tanpa mengubah konfigurasi keamanan server [cite: 1].

Ringkasan Hasil Pengujian

No.	Skenario Pengujian	Langkah Uji	Hasil yang Diharapkan	Status
1	Akses HTTPS	Buka https://100.110.137.96 di browser [cite: 1]	Aplikasi tampil, koneksi terenkripsi (sertifikat SSL) [cite: 1]	Berhasil [cite: 1]
2	Redirect HTTP ke HTTPS	Akses http://100.110.137.96 [cite: 1]	Otomatis redirect ke HTTPS (301) [cite: 1]	Berhasil [cite: 1]

No.	Skenario Pengujian	Langkah Uji	Hasil yang Diharapkan	Status
3	SSH di port non-default	ssh -p 2222 ubuntu@100.110.137.96 [cite: 1]	Login berhasil menggunakan key pair [cite: 1]	Berhasil [cite: 1]
4	Blokir SSH port 22	ssh -p 22 ubuntu@100.110.137.96 [cite: 1]	Connection refused/timeout [cite: 1]	Berhasil [cite: 1]
5	Fail2Ban aktif	Login SSH gagal 5x berturut-turut [cite: 1]	IP sumber diblokir, fail2ban-client menunjukkan banned IP [cite: 1]	Berhasil [cite: 1]
6	DB tidak accessible dari Host	mysql -h 127.0.0.1 -P 3306 -uroot -p [cite: 1]	Connection refused (port tidak terbuka) [cite: 1]	Berhasil [cite: 1]
7	Docker container running	docker ps [cite: 1]	3 container (nginx, php, mysql) berstatus Up [cite: 1]	Berhasil [cite: 1]
8	Port Scanning (Nmap)	nmap -sV -p 1-65535 100.110.137.96 [cite: 1]	Hanya port 80, 443, 2222 yang open [cite: 1]	Berhasil [cite: 1]
9	Vulnerability Scanning (Nikto)	nikto -h https://100.110.137.96 -ssl [cite: 1]	Tidak ditemukan kerentanan kritis [cite: 1]	Berhasil [cite: 1]
10	HTTP Header Inspection (cURL)	curl -I http://100.110.137.96 [cite: 1]	Response 301 redirect ke HTTPS [cite: 1]	Berhasil [cite: 1]

Detail Pengujian

1. Pengujian HTTPS dan Sertifikat SSL

Tujuan: Memastikan koneksi ke web server terenkripsi menggunakan HTTPS dengan sertifikat SSL [cite: 1].

Langkah Pengujian:

1. Buka browser dan akses https://100.110.137.96 [cite: 1].
2. Klik ikon "Not secure" pada address bar browser [cite: 1].

3. Klik "Certificate is not valid" untuk melihat detail sertifikat [cite: 1].

Hasil:

- Aplikasi Laravel berhasil ditampilkan melalui koneksi HTTPS [cite: 1].
- Browser menampilkan peringatan "Not secure" karena sertifikat berjenis Self-Signed Certificate yang dibuat menggunakan OpenSSL (bukan dari Certificate Authority publik) [cite: 1].
- Detail sertifikat menunjukkan Issuer: Internet Widgits Pty Ltd, membuktikan sertifikat berhasil di-generate dan aktif digunakan oleh Nginx [cite: 1].
- Koneksi tetap terenkripsi penuh meskipun browser menampilkan peringatan (hal ini normal untuk Self-Signed Certificate di lingkungan pengembangan/praktikum) [cite: 1].

Konfigurasi terkait (docker/nginx/default.conf):

```
server {  
    listen 443 ssl;  
    ssl_certificate /etc/ssl/certs/server.crt;  
    ssl_certificate_key /etc/ssl/private/server.key;  
}
```

2. Pengujian Redirect HTTP ke HTTPS

Tujuan: Memastikan semua akses HTTP otomatis dialihkan ke HTTPS [cite: 1].

Langkah Pengujian:

1. Buka browser dan akses `http://100.110.137.96` (tanpa "s") [cite: 1].
2. Perhatikan apakah URL otomatis berubah menjadi `https://` [cite: 1].

Hasil:

- Saat mengakses `http://100.110.137.96`, browser otomatis dialihkan (redirect 301) ke `https://100.110.137.96` [cite: 1].
- Ini membuktikan bahwa konfigurasi force redirect pada Nginx berjalan dengan benar sehingga tidak ada trafik yang lewat tanpa enkripsi [cite: 1].

Konfigurasi terkait (docker/nginx/default.conf):

```
server {  
    listen 80;  
    server_name _;  
    return 301 https://$host$request_uri;  
}
```

3. Pengujian SSH di Port Non-Default (2222)

Tujuan: Memastikan SSH hanya dapat diakses melalui port kustom 2222 (bukan port default 22) dan hanya menerima autentikasi berbasis key pair [cite: 1].

Langkah Pengujian:

1. Dari komputer lokal, jalankan perintah:
`ssh -p 2222 ubuntu@100.110.137.96` [cite: 1]
2. Verifikasi bahwa login berhasil tanpa memasukkan password (menggunakan SSH key

pair) [cite: 1].

Hasil:

- Login SSH berhasil dilakukan melalui port 2222 [cite: 1].
- Autentikasi menggunakan Public Key (tanpa password), sesuai konfigurasi PasswordAuthentication no [cite: 1].

Verifikasi konfigurasi efektif:

```
sudo sshd -T | grep -E  
'^(port|permitrootlogin|passwordauthentication) '
```

Output:

```
port 2222  
permitrootlogin no  
passwordauthentication no
```

4. Pengujian Blokir SSH Port Default (22)

Tujuan: Memastikan port SSH default (22) diblokir oleh firewall UFW sehingga tidak bisa diakses dari luar [cite: 1].

Langkah Pengujian:

1. Dari komputer lokal, jalankan perintah:
`ssh -p 22 ubuntu@100.110.137.96` [cite: 1]
2. Tunggu respons dari server [cite: 1].

Hasil:

- Koneksi ditolak (Connection refused / Connection timed out) [cite: 1].
- Ini membuktikan bahwa UFW berhasil memblokir akses ke port 22. Penyerang tidak dapat melakukan brute force pada port SSH default [cite: 1].

Verifikasi status UFW:

```
sudo ufw status verbose
```

Output:

```
Status: active Loggings: on (low)  
Default: deny (incoming), allow (outgoing), disabled (routed)
```

To	Action	From
--	-----	----
2222/tcp	ALLOW IN	Anywhere
80/tcp	ALLOW IN	Anywhere
443/tcp	ALLOW IN	Anywhere

Port 22 tidak ada dalam daftar rule, artinya semua koneksi ke port 22 otomatis ditolak oleh policy deny incoming [cite: 1].

5. Pengujian Fail2Ban (Brute Force Protection)

Tujuan: Memastikan Fail2Ban aktif memantau percobaan login SSH yang gagal dan memblokir IP penyerang secara otomatis [cite: 1].

Langkah Pengujian:

1. Dari komputer lokal, lakukan percobaan login SSH sebanyak 5 kali dengan password yang salah:
`ssh -p 2222 root@100.110.137.96`[cite: 1]
2. Setelah percobaan ke-5, cek apakah IP terblokir [cite: 1].
3. Verifikasi status Fail2Ban dari terminal VM:
`sudo fail2ban-client status sshd`[cite: 1]

Hasil:

- Setelah 5 kali percobaan login gagal, IP penyerang otomatis diblokir selama 1 jam (3600 detik) [cite: 1].
- Fail2Ban menunjukkan jail sshd dalam status aktif dan memantau log autentikasi [cite: 1].

Konfigurasi terkait (/etc/fail2ban/jail.local):

```
[sshd]
enabled = true
port = 2222
filter = sshd
logpath = /var/log/auth.log
maxretry = 5
bantime = 3600
```

6. Pengujian Isolasi Database (Port Tidak Terbuka ke Host)

Tujuan: Memastikan port database MySQL (3306) tidak dapat diakses dari luar container Docker (hanya bisa diakses secara internal oleh container aplikasi) [cite: 1].

Langkah Pengujian:

1. Dari terminal VM (host), jalankan perintah:
`mysql -h 127.0.0.1 -P 3306 -uroot -p`[cite: 1]

Hasil:

- Koneksi gagal dengan pesan Connection refused atau Command 'mysql' not found [cite: 1].
- Ini membuktikan bahwa port 3306 tidak diekspos ke host OS, sehingga database hanya dapat diakses oleh container aplikasi melalui jaringan internal Docker (backend network) [cite: 1].

Konfigurasi terkait (docker/docker-compose.yml):

```
db:
  image: mysql:8.0
  container_name: db_server
  networks:
    - backend
  # Port 3306 TIDAK di-mapping ke host
```

```
networks:
  backend:
    driver: bridge
    internal: true # Network ini tidak bisa akses internet
```

Perhatikan bahwa service db tidak memiliki properti ports, artinya port-nya hanya terbuka secara internal di dalam Docker network [cite: 1].

7. Pengujian Status Docker Container

Tujuan: Memastikan semua container berjalan dengan benar dan hanya container Nginx yang mengekspos port ke dunia luar [cite: 1].

Langkah Pengujian:

1. Dari terminal VM, jalankan perintah:

```
docker ps[cite: 1]
```

Hasil:

Container Name	Image	Status	Ports
reverse_proxy	nginx:alpine	Up	0.0.0.0:80->80, 0.0.0.0:443->443 [cite: 1]
app_server	(custom build)	Up	9000/tcp (internal only) [cite: 1]
db_server	mysql:8.0	Up	3306/tcp (internal only) [cite: 1]

- Hanya container reverse_proxy (Nginx) yang mempunyai port mapping ke 0.0.0.0 (terbuka ke publik) [cite: 1].
- Container app_server dan db_server hanya menampilkan port internal tanpa mapping ke host, membuktikan port isolation berjalan dengan benar [cite: 1].

Verifikasi Docker Network:

```
sudo docker network ls
```

Menampilkan dua network terpisah [cite: 1]:

- **frontend** – Menghubungkan Nginx ↔ App Server [cite: 1]
- **backend** – Menghubungkan App Server ↔ Database (bersifat internal: true) [cite: 1]

8. Pengujian Port Scanning dengan Nmap

Tujuan: Menggunakan tool Nmap untuk memindai port-port yang terbuka pada server dan memverifikasi bahwa hanya port yang diizinkan (80, 443, 2222) yang dapat diakses dari luar [cite: 1].

Langkah Pengujian:

1. Install Nmap pada komputer lokal (jika belum terinstal):

```
sudo apt install nmap[cite: 1]
```

2. Jalankan pemindaian port terhadap IP server:

```
nmap -sV -p 1-65535 100.110.137.96[cite: 1]
```

Hasil yang Diharapkan:

PORT	STATE	SERVICE	VERSION
22/tcp	filtered	ssh	
80/tcp	open	http	nginx
443/tcp	open	ssl/http	nginx
2222/tcp	open	ssh	OpenSSH 8.9p1 Ubuntu
3306/tcp	filtered	mysql	

Analisis:

- Port 80 (HTTP) dan 443 (HTTPS) berstatus open → sesuai konfigurasi UFW, port ini memang diizinkan untuk melayani traffic web [cite: 1].
- Port 2222 (SSH) berstatus open → ini adalah port SSH kustom yang sengaja dibuka untuk administrasi remote [cite: 1].
- Port 22 (SSH default) berstatus filtered → firewall UFW berhasil memblokir port SSH default. Penyerang tidak dapat mendeteksi service SSH di port standar [cite: 1].
- Port 3306 (MySQL) berstatus filtered → database tidak dapat diakses dari luar. Port ini hanya tersedia di dalam jaringan internal Docker [cite: 1].
- Seluruh port lainnya (1-65535) berstatus closed atau filtered → tidak ada port yang bocor/terbuka tanpa izin [cite: 1].

Kesimpulan Nmap: Hasil pemindaian membuktikan bahwa konfigurasi firewall UFW dan Docker port isolation bekerja dengan baik. Attack surface server diminimalkan hanya pada 3 port yang diperlukan [cite: 1].

9. Pengujian Web Vulnerability dengan Nikto

Tujuan: Menggunakan tool Nikto untuk melakukan pemindaian kerentanan (vulnerability scanning) pada web server dan mengidentifikasi potensi celah keamanan [cite: 1].

Langkah Pengujian:

1. Install Nikto pada komputer lokal (jika belum terinstal):

```
sudo apt install nikto[cite: 1]
```
2. Jalankan pemindaian terhadap web server:

```
nikto -h https://100.110.137.96 -ssl -nointeractive[cite: 1]
```

Hasil yang Diharapkan:

```
- Nikto v2.5.0
+ Target IP: 100.110.137.96
+ Target Hostname: 100.110.137.96
+ Target Port: 443
+ SSL Info: Subject:
/C=ID/ST=Indonesia/L=Bandung/O=Kelompok03KSJ Ciphers:
TLS_AES_256_GCM_SHA384
Issuer:
/C=ID/ST=Indonesia/L=Bandung/O=Kelompok03KSJ
+ Server: nginx
+ /: The site uses TLS and the Strict-Transport-Security HTTP header
```

```
is not defined.  
+ /: An alt-svc header was found which is advertising HTTP/3.  
+ No CGI Directories found.  
+ nginx appears to be outdated (current is at least 1.25.5)  
+ 1 host(s) tested
```

Analisis:

- SSL aktif: Nikto mengkonfirmasi bahwa server menggunakan TLS/SSL dengan sertifikat yang dikeluarkan oleh organisasi Kelompok03KSJ [cite: 1].
- Server header: Nginx terdeteksi sebagai web server (ini normal) [cite: 1].
- Strict-Transport-Security (HSTS): Header ini belum dikonfigurasi — ini adalah rekomendasi tambahan untuk penguatan keamanan di masa mendatang (tidak wajib untuk tugas ini) [cite: 1].
- Tidak ditemukan kerentanan kritis: Nikto tidak menemukan celah keamanan serius seperti directory listing, file sensitif yang terbuka, atau kerentanan injeksi [cite: 1].

Kesimpulan Nikto: Web server tidak memiliki kerentanan kritis. Temuan yang ada bersifat informatif (informational) dan tidak berdampak langsung pada keamanan sistem [cite: 1].

10. Pengujian HTTP Header dan Redirect dengan cURL

Tujuan: Menggunakan tool cURL untuk memverifikasi header HTTP response, memastikan redirect HTTP→HTTPS berjalan, dan mengecek informasi server [cite: 1].

Langkah Pengujian:

- Cek redirect HTTP ke HTTPS:
`curl -I http://100.110.137.96`[cite: 1]
- Cek header HTTPS:
`curl -Ik https://100.110.137.96`[cite: 1]

Hasil yang Diharapkan (HTTP → Redirect):

```
HTTP/1.1 301 Moved Permanently  
Server: nginx  
Location: https://100.110.137.96/
```

Hasil yang Diharapkan (HTTPS):

```
HTTP/1.1 200 OK  
Server: nginx  
Content-Type: text/html; charset=UTF-8  
Connection: keep-alive
```

Analisis:

- HTTP Response 301: Mengonfirmasi bahwa semua request HTTP dialihkan ke HTTPS secara permanen [cite: 1].
- HTTPS Response 200: Server merespons dengan sukses melalui koneksi terenkripsi [cite: 1].
- Server: nginx: Mengidentifikasi Nginx sebagai reverse proxy yang menangani request [cite: 1].

Kesimpulan cURL: Redirect dan enkripsi HTTPS berfungsi dengan benar sesuai konfigurasi [cite: 1].

Kesimpulan

Berdasarkan hasil pengujian di atas menggunakan kombinasi pengujian manual, Nmap (port scanning), Nikto (vulnerability scanning), dan cURL (header inspection), seluruh mekanisme keamanan yang diimplementasikan pada sistem web server Kelompok 03 telah berfungsi dengan baik dan sesuai rubrik penilaian, meliputi [cite: 1]:

- **Enkripsi HTTPS (SSL/TLS):** Koneksi terenkripsi menggunakan Self-Signed Certificate yang aktif di Nginx. Diverifikasi melalui browser, Nikto, dan cURL [cite: 1].
- **Redirect HTTP ke HTTPS:** Semua trafik HTTP otomatis dialihkan ke HTTPS (301). Diverifikasi melalui browser dan cURL [cite: 1].
- **SSH Hardening:** Port SSH dipindahkan ke 2222, root login dinonaktifkan, dan autentikasi hanya menerima key pair [cite: 1].
- **Firewall UFW:** Hanya port 80 (HTTP), 443 (HTTPS), dan 2222 (SSH) yang diizinkan masuk. Seluruh port lainnya diblokir. Diverifikasi melalui Nmap [cite: 1].
- **Fail2Ban:** Memblokir IP penyerang secara otomatis setelah 5 kali percobaan login gagal, dengan durasi ban selama 1 jam [cite: 1].
- **Isolasi Database:** Port database (3306) tidak diekspos ke host. Diverifikasi melalui Nmap (status filtered) dan docker ps [cite: 1].
- **Port Isolation Docker:** Hanya Nginx (Reverse Proxy) yang mengekspos port ke publik, sementara container App and Database sepenuhnya tersembunyi [cite: 1].
- **Nmap Port Scanning:** Hanya 3 port terbuka (80, 443, 2222), membuktikan attack surface yang minimal [cite: 1].
- **Nikto Vulnerability Scanning:** Tidak ditemukan kerentanan kritis pada web server [cite: 1].
- **cURL Header Inspection:** Redirect dan response header berfungsi sesuai standar keamanan [cite: 1].

Semua pengujian menghasilkan status **Berhasil**, menunjukkan bahwa sistem telah memenuhi standar keamanan yang ditetapkan untuk tugas ini [cite: 1].